Introduction

Duplicate messages are an inevitable aspect of distributed messaging with Kafka. Ensuring your application is able to handle these is essential. Using the Idempotent Consumer pattern coupled with the Transactional Outbox pattern is a recommended approach to achieve this. This article looks at these patterns and how to implement them.

Duplicate Message Scenarios

There is always a risk that your consumer will receive a duplicate message in a distributed system. There are different failure scenarios and configurations that can lead to this, impacted by how the application is designed and architected. This is covered in detail in the Deduplication Patterns article.

Duplicate messages can leave resources in unexpected and unintended states, as the duplicate events cause a resource to be processed multiple times. A database record may be updated multiple times, or multiple new entities created. External systems may be called multiple times. Events may be published multiple times to confirm an action has completed, causing further problems downstream.

Two example scenarios that can lead to duplicate messages are a consumer poll timeout, and a producer publishing duplicate events to a topic.

Consumer Poll Timeout

When a consumer polls for a batch of records it has a timeout within which it should process the batch and poll for the next batch of records. With the Java Apache Kafka client, this timeout is configured with the max.poll.interval.ms, and the maximum number of records to process in this period is configured with the max.poll.records.

Should the consumer instance not complete processing the batch within the timeout then the Kafka Broker assumes the consumer has died. It removes the consumer instance from the consumer group and rebalances the partitions. The partition that was assigned to this instance is now reassigned to another instance in the consumer group. Since the batch of records that were polled by the first instance have not been marked as consumed (via an update to the consumer offsets topic), these are the records that are polled by the newly assigned consumer instance.

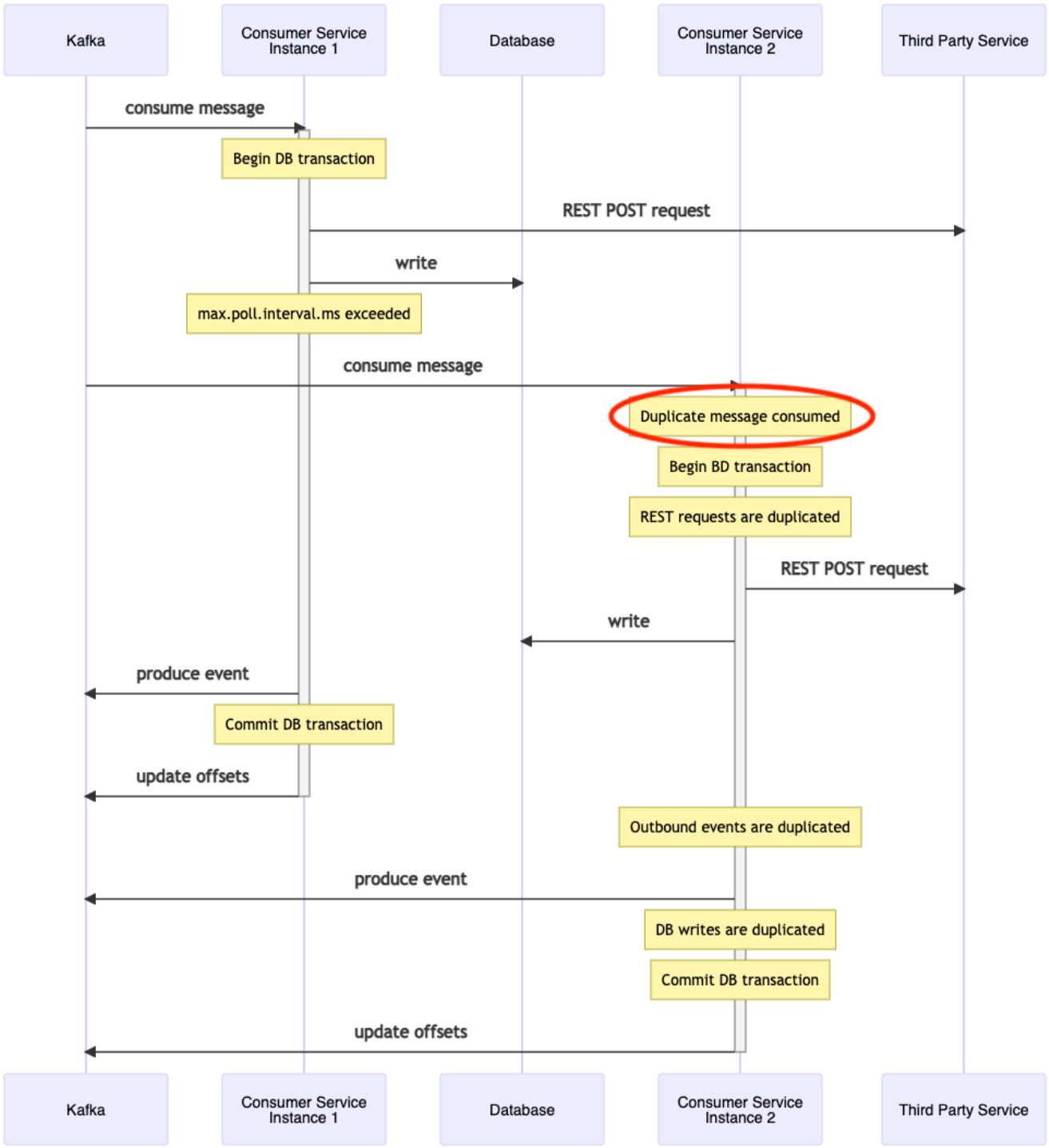

Figure 1: Duplicate Message Delivery

If the Kafka Broker did not take these actions and the consumer had in fact died, then the topic partitions that the consumer instance was listening to would be blocked.

While the new consumer instance is processing this batch of records, perhaps performing REST calls to other services, publishing new events to Kafka, and writing to the database, the first consumer instance is doing the same. If any of those actions were not idempotent, then data will be left in an inconsistent state across the system.

Non-Idempotent Producer

A Producer that is not configured to be idempotent can result in writing duplicate events to Kafka which would be consumed by listening services. The scenario where this happens is with a Producer writing to the topic but due to a transient error it does not receive acknowledgement of the write. It therefore retries the write in case it had actually failed, but can instead result in a duplicate write. This is covered in detail in the Idempotent Producer article.

The Idempotent Consumer Pattern

An Idempotent Consumer pattern uses a Kafka consumer that can consume the same message any number of times, but only process it once. To implement the Idempotent Consumer pattern the recommended approach is to add a table to the database to track processed messages. Each message needs to have a unique messageId assigned by the producing service, either within the payload, or as a Kafka message header. When a new message is consumed the table is checked for the existence of this message Id. If present, then the message is a duplicate. The consumer updates its offsets to effectively mark the message as consumed to ensure it is not redelivered, and no further action takes place.

If the message Id is not present in the table then a database transaction is started and the message Id is inserted. The message is then processed performing the required business logic. Upon completion the transaction is committed.

Database Flush Strategy

When implementing the Idempotent Consumer pattern care must be taken to ensure that the correct flushing strategy is used. The default flush strategy is often the 'transactional write-behind' strategy, and is the default strategy when using Hibernate and calling the save() method. With this strategy the flush of the persistence context happens at the last possible moment.

In the case where two duplicate events are consumed and are being processed in parallel, then both are able to start a transaction, write (uncommitted) the message Id to the database, and perform their message processing. Only when the second thread comes to commit the transaction would the constraint violation be thrown and the transaction be rolled back. This is illustrated in the following diagram, which has been simplified by removing the broker and 3PP service, and it is assumed that both consumer service instances have consumed the same duplicated message.

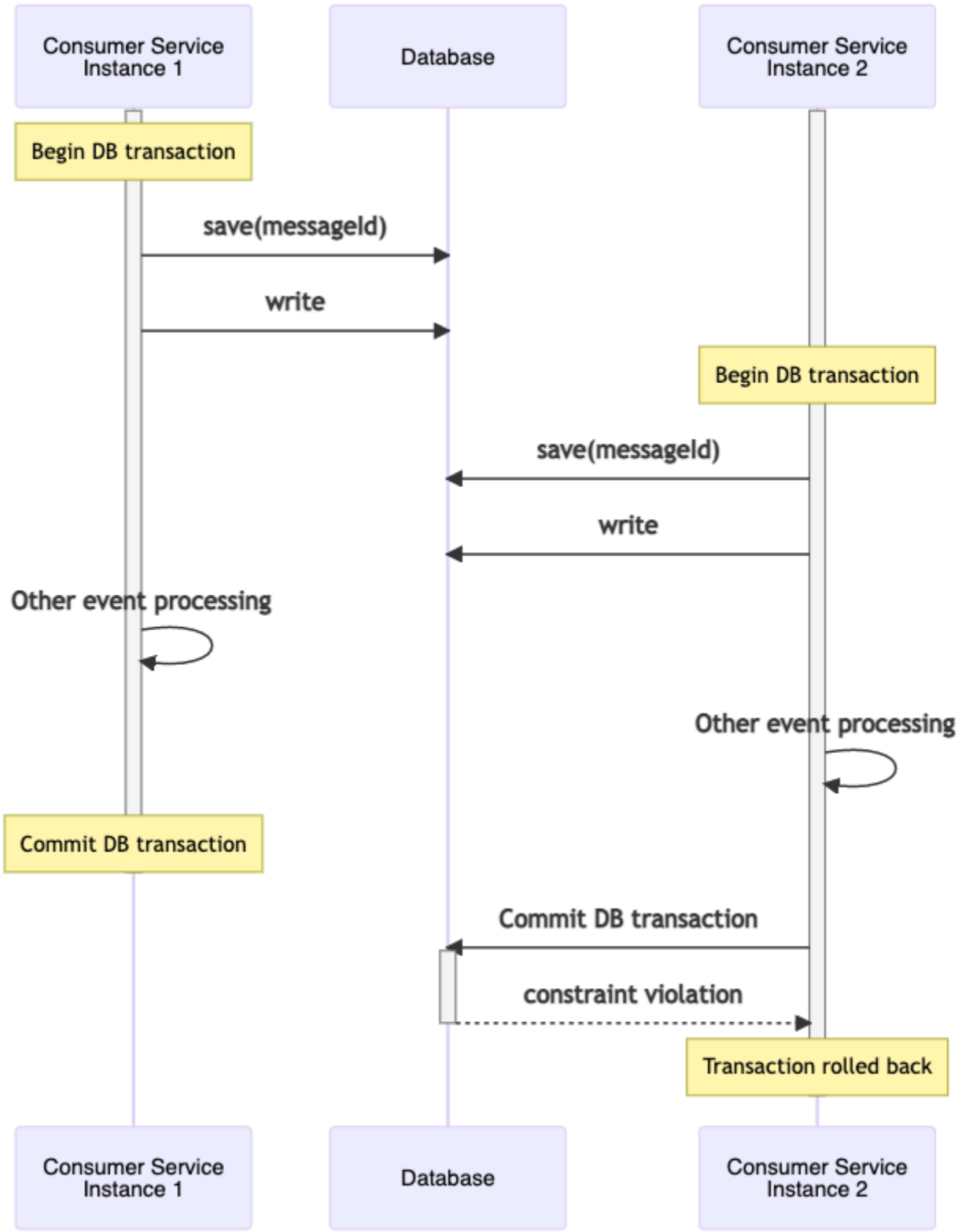

Figure 2: Idempotent Consumer

This could be acceptable, as the second database transaction would be rolled back once it tries to commit so any other database updates included in the transaction would be rolled back with this. With the Transactional Outbox pattern in place too (more on this below), there would be no risk of a duplicate resulting event being published.

However, any processing that cannot be rolled back would still be duplicated, such as a REST call to an external service, or events published without the Transactional Outbox pattern.

By employing a strategy where the data is flushed to the database at the point of save then this synchronizes the model state with the database (although it is still not committed). This results in a different duplicate handling flow. The second consumer instance consumes the duplicate event, starts a transaction, and attempts to write the message Id to the database. Now however it is not able to acquire a lock on the database row until the transaction started by the first consumer either commits or rolls back. If that first transaction commits successfully then at that point the second transaction is aborted. If the first transaction rolls back then the processing of the second duplicate is able to proceed, and itself can complete successfully or roll back.

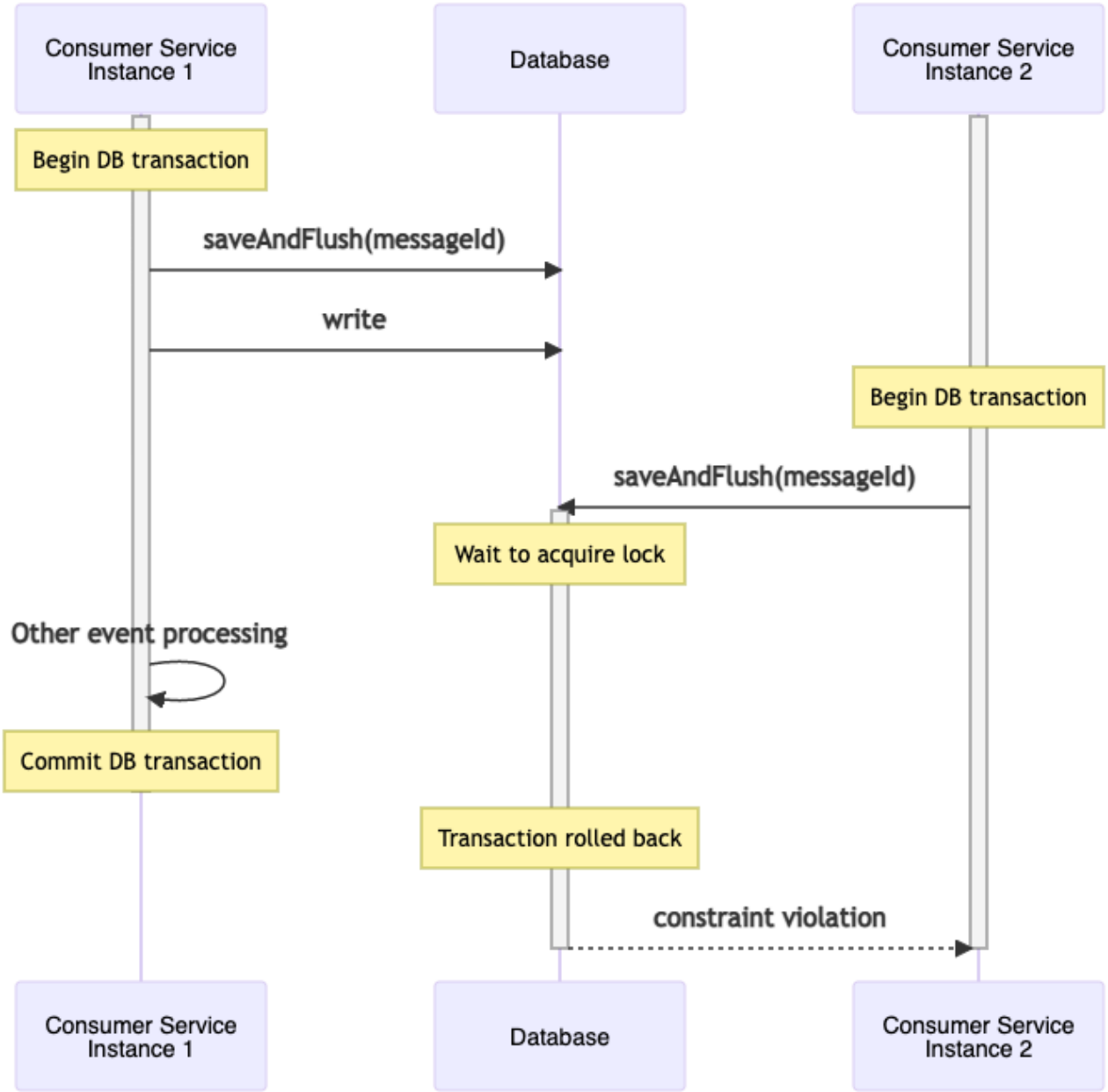

Figure 3: Idempotent Consumer - Data Flush Strategy

With Spring JPA and Hibernate it is simply a case of using the saveAndFlush() call when persisting the messageId within an **@Transactional** method.

```
private void deduplicate(UUID eventId) throws DuplicateEventException {
  try {
    processedEventRepository.saveAndFlush(new ProcessedEvent(eventId));
  } catch (DataIntegrityViolationException e) {
    throw new DuplicateEventException(eventId);
  }
}
```

This then is the recommended approach, with the advantage of minimising duplicate actions occurring. A simple scenario might be where the event is triggering the call to a third party bank rail service to send a payment. Without the flush strategy then two duplicate payments would be sent (which is illustrated by using save() instead of saveAndFlush()), with this strategy in place then just the one would.

If the first consumer failed before committing the database transaction, then the second consumer would now acquire the lock on the messageId and be able to process it.

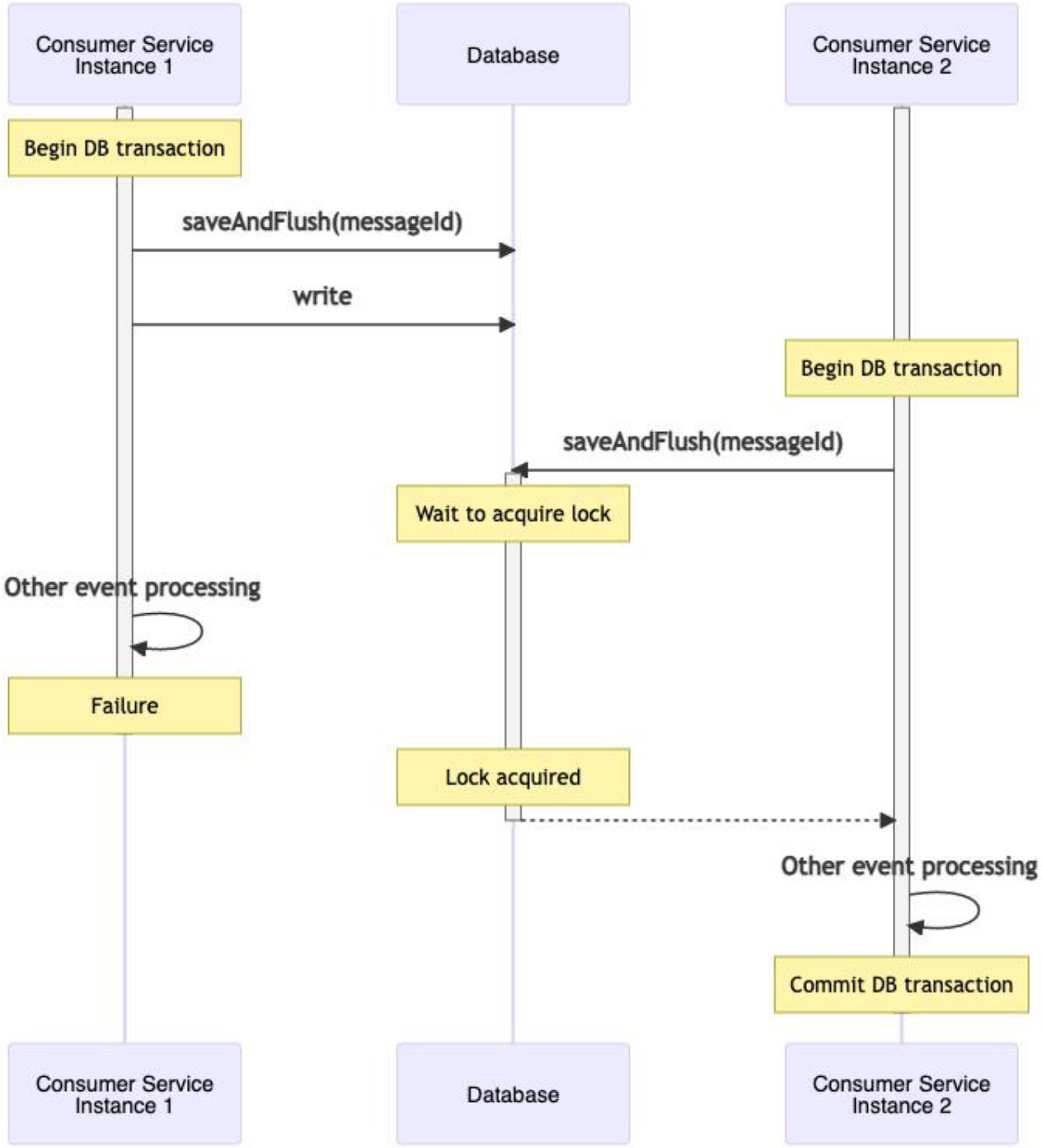

Figure 4: Idempotent Consumer - Failure Scenario

As a caveat it is important to remember that no matter what patterns are in place there is still the risk of a failure scenario that can lead to duplicate actions occurring, so the ramifications of this must be understood and catered for. This is covered in detail in the Deduplication Patterns article.

The Transactional Outbox Pattern

When a message is consumed and processed, resulting in an event being published to Kafka, implementing the Idempotent Consumer on its own is not sufficient to ensure this will not result in duplicate events being emitted.

While Kafka offers the option of Kafka Transactions, messages published to Kafka in a Kafka transaction are not atomic with resources written using database transactions. Therefore a failure occurring between a Database transaction being committed and a Kafka transaction being committed will leave resources in an inconsistent

state. Attempting to tie the commit of the two transactions together using a two-phase commit should also be avoided as this can fail under certain scenarios leaving the resources in an inconsistent state, adds complexity, and impacts performance by increasing the latency of all the transactions.

This can be addressed by effectively tying the publishing of an event to the database transaction by using the Transactional Outbox pattern. A new outbox table is created, and any event that is being published is inserted to this table.

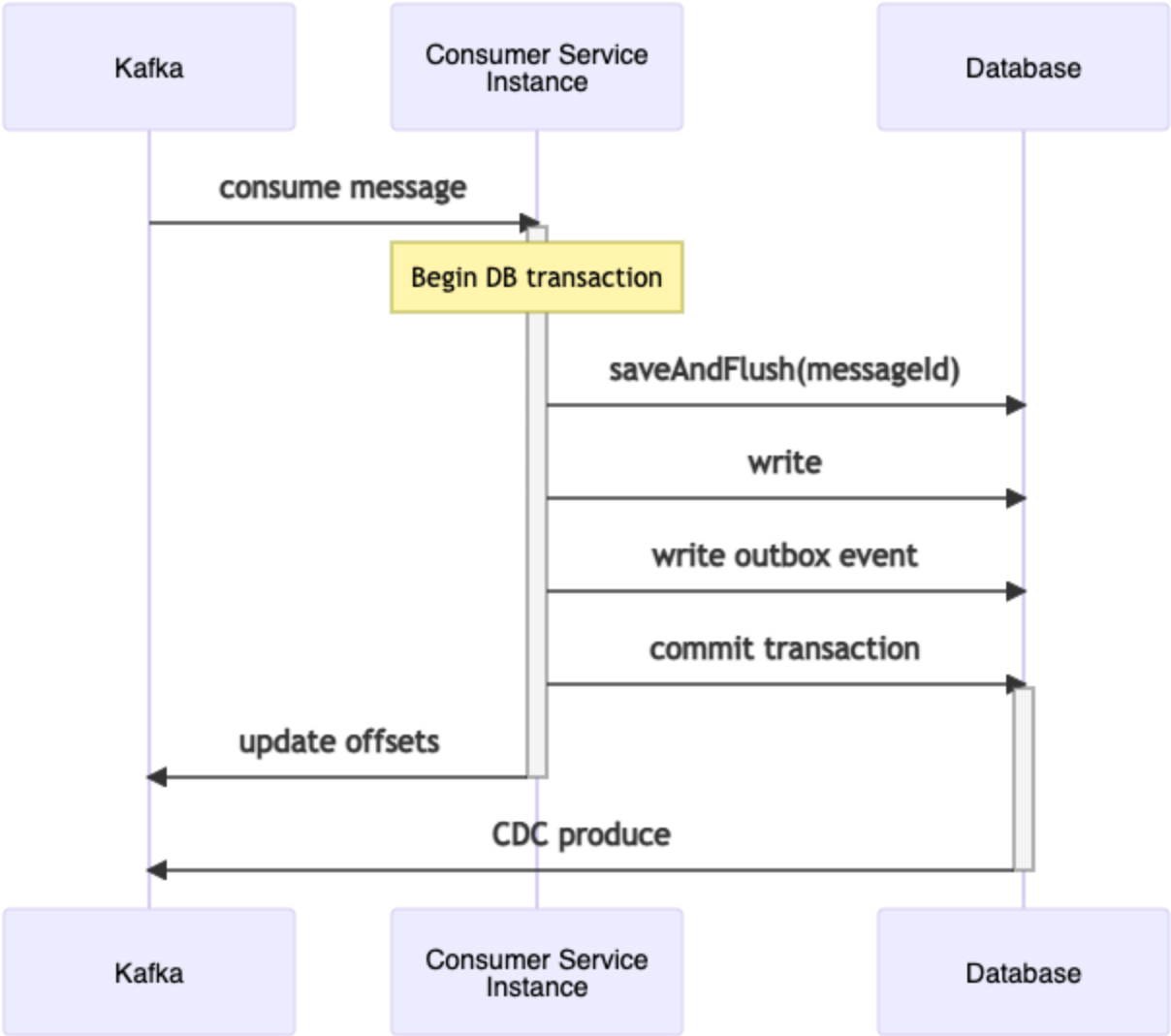

Figure 5: Idempotent Consumer & Transactional Outbox

The write of the outbound event to the outbox table happens as part of the same database transaction that includes the write of the consumed message Id by the Idempotent Consumer. This means that the consume of the message, any resulting outbound event publishing, and any other database writes, are all atomic. They either all succeed or all fail. This provides the strongest guarantee that no duplicate processing will occur.

CDC and Kafka Connect

Change Data Capture (CDC) with a Kafka Connect connector can then be used to write the event to the outbound Kafka topic. The outbox event is written to the database commit log, and the connector reads this, transforms the data, and writes it to the Kafka topic.

Kafka Connect is a Kafka component for providing simple integration between a datasource (in this case a database) and Kafka. An off the shelf connector such as Debezium could be employed here, connecting a Postgres or MySQL database to Kafka. This should only require configuration in order to use, from connecting to the source and sink datasources, simple message transforms, and error handling. The option to code and plug in more complex transforms is available, as is the option to code a new connector if necessary. Many connectors are available for different databases, so writing your own is usually not required, and would increase the application complexity.

There are other options for implementing the Transactional Outbox pattern besides using CDC. For example, a simple poller could be implemented to poll the outbox for new events, writing them to Kafka. Kafka Connect however comes with all the advantages that Kafka brings to the table such as resilience, fault tolerance, error handling, and scalability.

Data Consistency In Failure Scenarios

As the consumer offsets are only updated after the database transaction commits, then if the consumer dies after writing the outbox event but before the database transaction commits, then no outbound message is sent, and the message will be redelivered.

If the consumer dies after the database transaction commits but before the consumer offsets are written, then the message is redelivered, but deduplicated by the processed messageIds check, so no duplication occurs. In this case the consumer offsets must be updated by the second consumer to ensure the message is not continually redelivered.

Revisiting the original diagram (Figure 1) showing the duplicate message delivery, this time with the Idempotent Consumer and Transactional Outbox patterns in place:

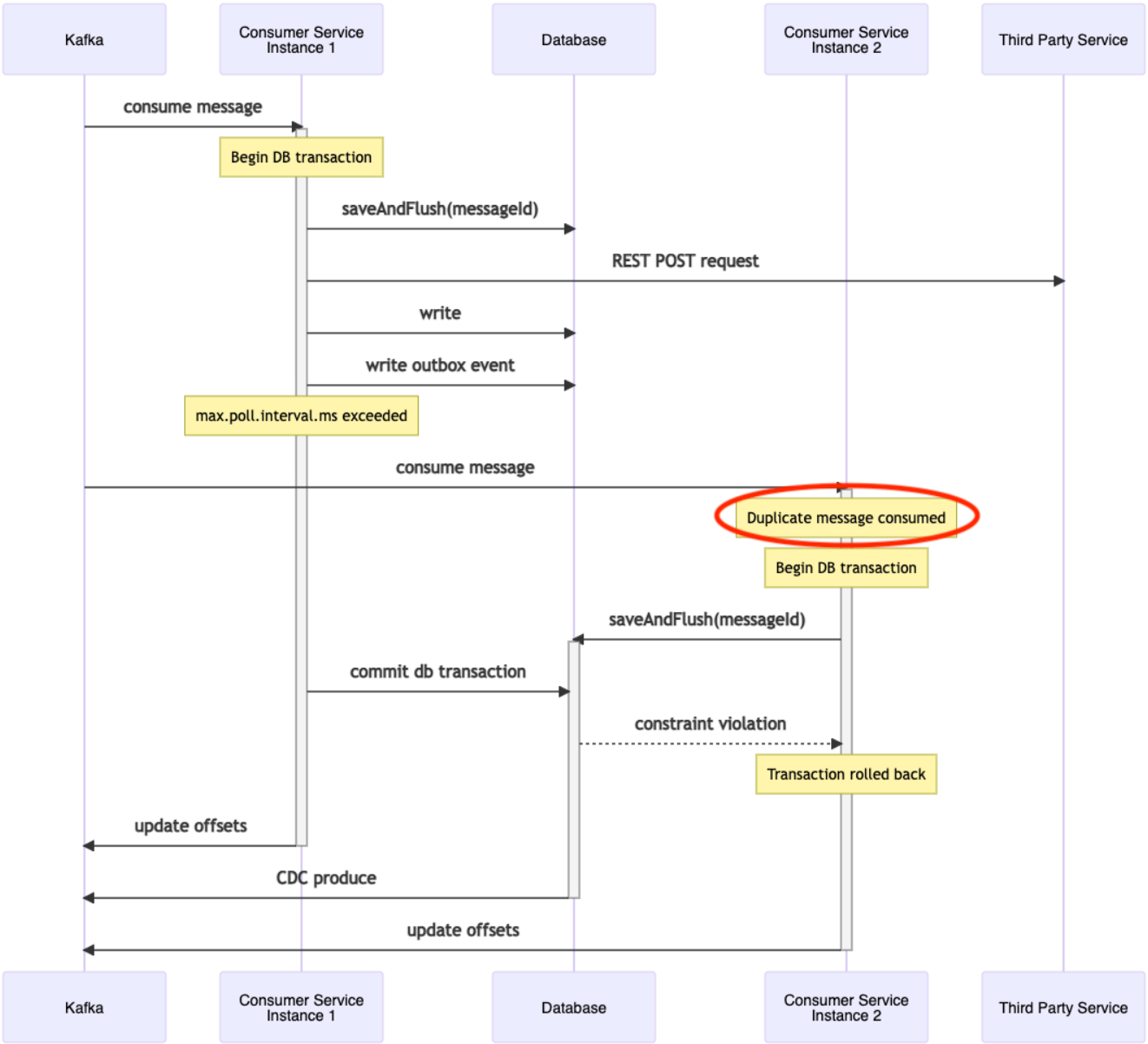

Figure 6: Idempotent Consumer & Transactional Outbox - Duplicate Delivery

Conclusion

Coupling the Idempotent Consumer using the data flush strategy with the Transactional Outbox and CDC provides the gold standard in managing duplicate messages in applications using Kafka for distributed messaging. Guaranteeing deduplication using these patterns comes at the cost of extra complexity and more components in place in the Architecture, so it will be a trade-off to determine whether the advantages out-weigh the disadvantages for each use case.

Source Code

The source code is available here: https://github.com/lydtechconsulting/kafka-idempotent-consumer

The project demonstrates event deduplication with the Idempotent Consumer pattern, and emitting events via the Transactional Outbox pattern. It uses Debezium as the Kafka Connect component for Change Data Capture (CDC) in order to publish outbound events.

It contains SpringBoot integration tests using the embedded Kafka Broker and H2 database that demonstrate event deduplication when duplicate events are consumed by the application.

It also contains component tests that use TestContainers to manage spinning up a dockerised Kafka broker, a dockerised Kafka Connect using Debezium for CDC, a dockerised Postgres database, and a dockerised wiremock to represent a third party service. This call to the third party service simulates transient errors that can be successful on retry. The delay caused by retry can cause duplicate message delivery, enabling demonstration of event deduplication.